Lecture 8: CLIP and Classifier-Free Guidance

텍스트-이미지 매핑과 조건부 생성의 핵심 기법

목차 (Table of Contents)

- 0. 개요 (Overview)
  - 0.1 선수 지식
  - 0.2 학습 목표
- 1. CLIP (Contrastive Language-Image Pre-training)
  - 1.1 핵심 아이디어
  - 1.2 학습: Contrastive Learning
  - 1.3 구조
  - 1.4 PyTorch 구현: CLIP 모델
  - 1.5 CLIP의 놀라운 능력: Zero-Shot Classification
  - 1.6 생성 모델에서 CLIP의 역할
- 2. 조건부 디퓨전: 텍스트를 이미지 생성에 주입하는 방법
  - 2.1 Cross-Attention — 표준 방법
  - 2.2 PyTorch 구현: Cross-Attention Layer
  - 2.3 AdaLN — DiT에서의 조건 주입
- 3. Classifier Guidance — 분류기를 이용한 안내
  - 3.1 아이디어
  - 3.2 한계
- 4. Classifier-Free Guidance (CFG)
  - 4.1 핵심 아이디어
  - 4.2 학습: 조건 드롭아웃
  - 4.3 생성: 가중 외삽
  - 4.4 수학적 해석
  - 4.5 기하학적 직관
  - 4.6 PyTorch 구현: CFG Sampling
  - 4.7 Negative Prompt
  - 4.8 PyTorch 구현: Negative Prompt
  - 4.9 Guidance Scale의 실용적 선택
- 5. 조건부 학습의 전체 파이프라인
  - 5.1 학습 시
  - 5.2 생성 시
- 6. FID와 CLIP Score의 Trade-off

    - 6.1 Guidance Scale에 따른 지표 변화
    - 6.2 Pareto Front
- 7. 조건부 생성의 다른 접근법들
    - 7.1 비교
    - 7.2 ControlNet (간략)
- 8. 핵심 수식 요약
- 9. 다음 강의 미리보기
- 참고문헌 (References)

---

0. 개요 (Overview)

DDPM(강의 7)은 **무조건부 생성**이었다: 랜덤 노이즈 → 랜덤 이미지. 하지만 실용적으로 원하는 것은 **"고양이를 그려줘"**라고 말하면 고양이가 나오는 **조건부 생성**이다.

> 💡 **왜 조건부 생성이 핵심 문제인가?** 무조건부 생성 모델은 학습 데이터의 분포 $p(x)$를 학습한다. 즉, "세상에 존재할 법한 이미지"를 랜덤하게 만들어낸다. 하지만 실제 응용에서 사용자는 **특정한** 이미지를 원한다. 이는 조건부 분포 $p(x \mid c)$를 모델링해야 한다는 뜻이다. 여기서 c는 텍스트, 클래스 레이블, 참조 이미지 등 다양한 형태의 조건(condition)이 될 수 있다. 무조건부 모델 $p(x)$에서 조건부 모델 $p(x \mid c)$로의 전환은 단순히 "입력을 하나 더 받는 것"이 아니라, 근본적으로 다른 두 가지 문제를 해결해야 한다: (1) 조건 c를 모델이 이해할 수 있는 형태로 인코딩하는 것 (CLIP이 해결), (2) 인코딩된 조건을 생성 과정에 효과적으로 반영하는 것 (CFG가 해결).

이 강의에서는 조건부 생성의 두 가지 핵심 구성 요소를 다룬다:

구성 요소	역할	년도
CLIP	텍스트와 이미지를 같은 공간에 매핑하는 인코더	2021
Classifier-Free Guidance	조건을 강하게 따르도록 하는 생성 기법	2022

이 두 구성 요소는 서로 독립적으로 개발되었지만, 결합되면서 현대 text-to-image 생성의 근간이 되었다. CLIP은 "텍스트와 이미지가 같은 의미인지 판단"하는 능력을, CFG는 "그 판단을 생성 과정에서 증폭"하는 능력을 제공한다.

0.1 선수 지식

이 강의를 이해하기 위해 다음 개념에 대한 사전 지식이 필요하다:

선수 지식	관련 강의/주제	핵심 개념

DDPM	강의 7	Forward/Reverse Process, 노이즈 예측, ϵ-prediction
Transformer Attention	강의 5	Self-Attention, Multi-Head Attention, Query/Key/Value
Contrastive Learning	기초 ML	InfoNCE Loss, Positive/Negative pair, 유사도 함수
Vision Transformer (ViT)	강의 5	Patch embedding, [CLS] token, Position encoding

0.2 학습 목표

이 강의를 마치면 다음을 할 수 있다:

1. CLIP의 contrastive learning 메커니즘을 설명하고 PyTorch로 구현할 수 있다
2. Cross-Attention을 통해 텍스트 조건을 디퓨전 모델에 주입하는 방법을 이해한다
3. Classifier Guidance와 Classifier-Free Guidance의 수학적 차이를 비교할 수 있다
4. CFG sampling 알고리즘을 구현하고, guidance scale w의 효과를 분석할 수 있다
5. Negative Prompt의 작동 원리를 이해하고 활용할 수 있다
6. FID-CLIP Score trade-off를 이해하고 적절한 w를 선택할 수 있다

---

1. CLIP (Contrastive Language-Image Pre-training)

> 📄 Radford et al., "Learning Transferable Visual Models From Natural Language Supervision", OpenAI, 2021

1.1 핵심 아이디어

텍스트와 이미지를 **같은 벡터 공간**에 넣는다. "고양이 사진"과 "a photo of a cat"이라는 텍스트가 가까운 벡터로 매핑되도록 학습.

```
"a photo of a cat" → [Text Encoder]  → z_text ∈ ℝ^768
                                          ↕ 가깝게
고양이 이미지       → [Image Encoder] → z_image ∈ ℝ^768
```

> 💡 **왜 이런 접근이 필요한가?** CLIP 이전에 텍스트와 이미지를 연결하는 방법은 크게 두 가지였다:
>
> **(1) 생성적 접근 (Generative approach)**: 이미지가 주어지면 캡션을 생성하거나 (Image Captioning), 캡션이 주어지면 이미지를 생성한다. 이 방식은 전체 출력 분포를 모델링해야 하므로 매우 비용이 크다. 예를 들어, 이미지에서 텍스트를 생성하려면 자연어의 방대한 분포를 모델링해야 하고, 텍스트에서 이미지를 생성하려면 모든 픽셀의 분포를 모델링해야 한다.

> **(2) 분류적 접근 (Classification approach)**: ImageNet처럼 이미지를 미리 정해진 N개의 카테고리로 분류한다. 이 방식은 확장성이 없다 -- 새로운 개념이 등장하면 레이블 체계를 다시 만들고 모델을 재학습해야 한다.
>
> **CLIP의 혁신**: 생성도, 분류도 하지 않는다. 대신 **"이 텍스트와 이 이미지가 매칭되는가?"**라는 훨씬 단순한 질문에만 답하도록 학습한다. 이것이 **contrastive learning (대조 학습)**이다. 매칭 여부만 판단하면 되므로 (1) 픽셀이나 단어를 하나하나 생성할 필요가 없어 스케일링이 쉽고, (2) 사전에 카테고리를 정의할 필요가 없어 개방적이다. 인터넷에서 수집한 4억 개의 (이미지, 캡션) 쌍만 있으면 된다.

⚠️ **중요**: CLIP은 생성 모델이 아니라 **표현 학습(representation learning)** 모델이다. 이미지를 만들지 않고, 텍스트와 이미지 사이의 의미적 관계를 학습한다. 표현 학습이란, 입력 데이터(이미지, 텍스트)를 의미를 잘 포착하는 벡터(representation)로 변환하는 함수를 학습하는 것이다. 좋은 표현이란 의미적으로 유사한 것들이 벡터 공간에서도 가까이 위치하는 표현을 말한다.

1.2 학습: Contrastive Learning

> 💡 **왜 Contrastive Learning인가?** Contrastive learning (대조 학습)의 핵심 아이디어는 **"비교를 통해 학습한다"**는 것이다. "고양이가 무엇인지"를 직접 정의하는 대신, "이 이미지와 이 텍스트가 서로 맞는 쌍인가, 아닌가?"를 판단하는 능력을 학습한다. 이 접근법이 강력한 이유는 다음과 같다:
>
> 1. **레이블링이 필요 없다**: 인터넷에서 이미지와 그 캡션을 긁어오기만 하면 자동으로 (이미지, 텍스트) 쌍이 만들어진다. 사람이 일일이 "이것은 고양이"라고 분류할 필요가 없다.
> 2. **확장성이 뛰어나다**: 생성 모델처럼 모든 픽셀을 복원하거나 모든 단어를 생성할 필요가 없다. 두 벡터의 유사도만 계산하면 되므로 계산 비용이 상대적으로 낮다.
> 3. **개방적 어휘를 다룬다**: 사전에 카테고리를 정의할 필요 없이, 자유로운 자연어 텍스트를 그대로 사용한다.

배치 내 N개의 (이미지, 텍스트) 쌍이 주어질 때:

$$\text{유사도 행렬: } S_{ij} = \frac{z_i^{\text{img}} \cdot z_j^{\text{txt}}}{\|z_i^{\text{img}}\| \cdot \|z_j^{\text{txt}}\|} \cdot \exp(\tau)$$

여기서 $\frac{z_i^{\text{img}} \cdot z_j^{\text{txt}}}{\|z_i^{\text{img}}\| \cdot \|z_j^{\text{txt}}\|}$는 **코사인 유사도(cosine similarity)**이다.

> 💡 **왜 dot product가 아니라 코사인 유사도(cosine similarity)를 사용하는가?** 단순 dot product $z_i \cdot z_j$는 벡터의 **방향**뿐만 아니라 **크기(magnitude)**에도 영향을 받는다. 만약 이미지 인코더가 특정 이미지에 대해 큰 크기의 벡터를 출력하면, 그 이미지는 모든 텍스트와 높은 유사도를 가지게 된다 -- 의미적으로 관련이 없더라도! 코사인 유사도는 두 벡터를 L2 정규화(단위 벡터로 만든 뒤)한 후의 내적이므로, 순수하게 **방향(angle)만으로** 유사도를 측정한다.

> 기하학적으로, L2 정규화는 모든 임베딩을 **단위 초구(unit hypersphere)** 위에 놓는 것이다. 768차원 공간의 단위 구 표면 위에서, 의미적으로 유사한 것들은 가까이, 다른 것들은 멀리 배치된다. 이렇게 하면 한쪽 modality(이미지 또는 텍스트)가 다른 modality를 지배하는 것(domination)을 방지할 수 있다.

- 대각선 (i, i): **맞는 쌍 (positive pair)** → 유사도를 높여야 함
- 비대각선 (i, j), $i \neq j$: **틀린 쌍 (negative pair)** → 유사도를 낮춰야 함

그리고 τ는 **학습 가능한 temperature parameter (온도 파라미터)**이다.

> 💡 **Temperature τ는 왜 필요하고, 왜 학습하는가?**
>
> Temperature는 softmax 분포의 **날카로움(sharpness)**을 제어한다. softmax 함수에 들어가기 전에 logit을 $\exp(\tau)$로 스케일링한다고 생각하면 된다:
>
> - **τ가 클 때 (높은 온도)**: $\exp(\tau)$가 커져서 logit의 차이가 증폭된다. softmax 출력이 **날카로워져서** 가장 유사한 쌍에 확률이 집중된다. 모델이 더 **자신감 있는(confident)** 예측을 한다.
> - **τ가 작을 때 (낮은 온도)**: $\exp(\tau)$가 작아져서 logit의 차이가 줄어든다. softmax 출력이 **부드러워져서** 확률이 고르게 분포한다. 모델이 더 **불확실한(uncertain)** 예측을 한다.
>
> 왜 τ를 고정하지 않고 **학습**하는가? 최적의 temperature는 학습 데이터의 난이도, 배치 크기, 임베딩 공간의 구조 등에 따라 달라진다. 학습 초기에는 임베딩이 아직 의미 없으므로 부드러운 분포가 유리하고, 학습이 진행되면서 임베딩이 의미를 가지게 되면 더 날카로운 분포가 유리하다. 모델이 τ를 스스로 조절하게 하면 이런 동적 변화에 자동으로 적응한다.
>
> CLIP 논문에서 τ는 log scale로 파라미터화되며 ($\log \tau$를 학습), 학습 중 상한을 두어 발산을 방지한다.

Image→Text 방향의 Loss:

$$\mathcal{L}_{\text{i2t}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(S_{ii})}{\sum_{j=1}^{N} \exp(S_{ij})}$$

Text→Image 방향의 Loss:

$$\mathcal{L}_{\text{t2i}} = -\frac{1}{N}\sum_{i=1}^{N} \log \frac{\exp(S_{ii})}{\sum_{j=1}^{N} \exp(S_{ji})}$$

전체 Loss:

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}(\mathcal{L}_{\text{i2t}} + \mathcal{L}_{\text{t2i}})$$

> 직관: $N \times N$ 유사도 행렬에서 대각선을 최대화하는 것은 N-way 분류 문제 2개를 동시에 푸는 것과 같다.

InfoNCE Loss의 이론적 배경: 상호 정보량과의 관계

위의 CLIP Loss는 사실 **InfoNCE Loss** (Noise Contrastive Estimation을 정보 이론에 적용한 것)의 한 형태이다. 이를 이론적으로 이해하기 위해 상호 정보량(mutual information)과의 관계를 단계별로 유도해보자.

Step 1: 상호 정보량의 정의

두 확률 변수 X (이미지)와 T (텍스트) 사이의 상호 정보량은 다음과 같다:

$$I(X;T) = \mathbb{E}_{p(x,t)}\left[\log \frac{p(x,t)}{p(x)p(t)}\right]$$

이것은 "X와 T가 독립일 때의 분포 $p(x)p(t)$"와 "실제 결합 분포 $p(x,t)$"가 얼마나 다른지를 KL divergence로 측정한 것이다. 상호 정보량이 높을수록 X를 알면 T에 대해 많은 것을 알 수 있다 (그 반대도 마찬가지).

Step 2: 왜 상호 정보량 최대화가 좋은 표현 학습인가?

$I(Z_{\text{img}}; Z_{\text{txt}})$를 최대화한다는 것은, 이미지 임베딩 Z_{img}가 텍스트 임베딩 Z_{txt}에 대한 정보를 최대한 많이 담도록 강제하는 것이다. 즉, 이미지 벡터만 보고도 그에 대응하는 텍스트가 무엇인지 최대한 알아낼 수 있도록 임베딩 공간을 구성한다.

Step 3: InfoNCE는 상호 정보량의 하한

Oord et al. (2018)이 증명한 핵심 결과는 다음과 같다:

$$I(X;T) \geq \log N - \mathcal{L}_{\text{InfoNCE}}$$

여기서 $\mathcal{L}_{\text{InfoNCE}}$는 CLIP Loss와 동일한 형태이다. 따라서 **InfoNCE Loss를 최소화하면 상호 정보량의 하한을 최대화**하게 된다.

직관적으로, 배치 크기 N이 클수록 하한이 더 타이트해진다 ($\log N$이 커지므로). 이것이 CLIP이 **배치 크기 32,768**이라는 극도로 큰 배치를 사용하는 근본적인 이유이다: 배치 크기가 곧 negative sample의 수이며, negative가 많을수록 상호 정보량을 더 정확하게 추정할 수 있다.

Step 4: Loss 계산의 구체적 예시

배치 크기 $N = 4$일 때의 유사도 행렬을 구체적으로 계산해보자:

```
                텍스트1  텍스트2  텍스트3  텍스트4
이미지1  [   0.9     0.1     0.2     0.05  ]   ← i=1에 대해 softmax -
이미지2  [   0.15    0.85    0.1     0.3   ]
이미지3  [   0.1     0.2     0.88    0.15  ]
이미지4  [   0.05    0.1     0.2     0.92  ]

대각선(볼드) = positive pair, 나머지 = negative pair
```

$\mathcal{L}_{\text{i2t}}$에서 $i = 1$의 기여:

$$-\log \frac{\exp(0.9)}{\exp(0.9) + \exp(0.1) + \exp(0.2) + \exp(0.05)}$$

분자는 positive pair의 유사도, 분모는 이미지1과 모든 텍스트의 유사도 합이다. 이 값을 최소화하면 $S_{11} = 0.9$은 더 높이고, S_{12}, S_{13}, S_{14}는 더 낮추게 된다.

양방향 Loss가 필요한 이유: $\mathcal{L}_{\text{i2t}}$만 사용하면 "각 이미지에 대해 올바른 텍스트를 찾는" 능력만 학습한다. $\mathcal{L}_{\text{t2i}}$를 추가해야 "각 텍스트에 대해 올바른 이미지를 찾는" 능력도 학습한다. 행렬의 행 방향과 열 방향 모두에서 대각선을 최대화해야 진정한 대칭적 정렬(alignment)이 이루어진다.

1.3 구조

구성 요소	구현	출력
Image Encoder	ViT-L/14 (Vision Transformer)	768차원 벡터
Text Encoder	Transformer (12-layer)	768차원 벡터
Projection	Linear layer → L2 정규화	단위 구 위의 벡터

학습 데이터: **4억 개**의 인터넷 텍스트-이미지 쌍 (WebImageText, WIT)

> 💡 **왜 이런 구조적 선택이 이루어졌는가?**
>
> **Image Encoder로 ViT를 선택한 이유**: CLIP 논문에서는 ResNet과 ViT 두 가지를 비교했다. 작은 모델에서는 ResNet이 효율적이지만, 모델 크기를 키울수록 ViT가 더 좋은 성능을 보였다. ViT는 Self-Attention으로 이미지 전체의 전역적(global) 관계를 한 번에 파악할 수 있기 때문이다. "ViT-L/14"에서 L은 Large 모델 크기를, 14는 패치 크기(14x14 픽셀)를 의미한다.
>
> **Text Encoder로 Transformer를 사용하는 이유**: GPT 스타일의 causal(자기회귀) Transformer를 사용한다. BERT 같은 양방향 모델도 가능하지만, causal mask를 사용하면 [EOS] (End of Sequence) 토큰의 출력이 전체 텍스트 의미를 요약하게 된다 -- 마지막 토큰이 모든 이전 토큰의 정보를 축적하므로 자연스러운 "요약 벡터"가 된다.
>
> **Projection Layer의 역할**: 이미지 인코더와 텍스트 인코더의 내부 표현 차원이 다를 수 있다 (예: 이미지 인코더의 내부 차원이 1024이고 텍스트 인코더가 512일 수 있음). Projection Layer는 이 둘을 **공유 임베딩 공간(shared embedding space)**의 같은 차원으로 매핑한다. 그 후 L2 정규화를 적용하여 모든 벡터를 단위 초구(unit hypersphere) 위에 놓는다. 이렇게 하면 내적이 곧 코사인 유사도가 되어 유사도 계산이 단순해진다.

1.4 PyTorch 구현: CLIP 모델

아래는 CLIP의 핵심 구조를 간략화한 PyTorch 구현이다:

```python
import torch
import torch.nn as nn
import torch.nn.functional as F


class CLIPImageEncoder(nn.Module):
    """CLIP 이미지 인코더 (간략화된 ViT 기반)"""

    def __init__(self, image_size=224, patch_size=14, embed_dim=768,
                 num_heads=12, num_layers=12):
        super().__init__()
        self.patch_size = patch_size
        num_patches = (image_size // patch_size) ** 2

        # 이미지를 패치로 분할하고 임베딩
        self.patch_embed = nn.Conv2d(
            3, embed_dim, kernel_size=patch_size, stride=patch_size
        )

        # [CLS] 토큰: 이미지 전체를 대표하는 벡터
        self.cls_token = nn.Parameter(torch.randn(1, 1, embed_dim))

        # 위치 인코딩
        self.pos_embed = nn.Parameter(
            torch.randn(1, num_patches + 1, embed_dim)
        )

        # Transformer 인코더 레이어들
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=num_heads,
            dim_feedforward=embed_dim * 4,
            activation='gelu', batch_first=True
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers
        )

        self.layer_norm = nn.LayerNorm(embed_dim)

        # 공유 임베딩 공간으로의 프로젝션
        self.projection = nn.Linear(embed_dim, embed_dim, bias=False

    def forward(self, images):
        """
        Args:
            images: [B, 3, 224, 224] 입력 이미지 텐서
        Returns:
            [B, embed_dim] L2 정규화된 이미지 임베딩
        """
        B = images.shape[0]

        # 패치 임베딩: [B, 3, 224, 224] → [B, embed_dim, H', W'] → [
```

```python
        x = self.patch_embed(images)  # [B, embed_dim, 16, 16]
        x = x.flatten(2).transpose(1, 2)  # [B, 256, embed_dim]

        # [CLS] 토큰 추가
        cls_tokens = self.cls_token.expand(B, -1, -1)
        x = torch.cat([cls_tokens, x], dim=1)  # [B, 257, embed_dim

        # 위치 인코딩 더하기
        x = x + self.pos_embed

        # Transformer 통과
        x = self.transformer(x)
        x = self.layer_norm(x)

        # [CLS] 토큰 출력만 사용 → 이미지 전체 표현
        x = x[:, 0]  # [B, embed_dim]

        # 프로젝션 후 L2 정규화
        x = self.projection(x)
        x = F.normalize(x, dim=-1)  # 단위 구 위의 벡터로 만듦

        return x


class CLIPTextEncoder(nn.Module):
    """CLIP 텍스트 인코더 (간략화된 Transformer 기반)"""

    def __init__(self, vocab_size=49408, max_seq_len=77,
                 embed_dim=768, num_heads=12, num_layers=12):
        super().__init__()

        # 토큰 임베딩 + 위치 임베딩
        self.token_embed = nn.Embedding(vocab_size, embed_dim)
        self.pos_embed = nn.Parameter(
            torch.randn(1, max_seq_len, embed_dim)
        )

        # Transformer 인코더 (causal mask 사용)
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=embed_dim, nhead=num_heads,
            dim_feedforward=embed_dim * 4,
            activation='gelu', batch_first=True
        )
        self.transformer = nn.TransformerEncoder(
            encoder_layer, num_layers=num_layers
        )

        self.layer_norm = nn.LayerNorm(embed_dim)
        self.projection = nn.Linear(embed_dim, embed_dim, bias=False

    def forward(self, token_ids):
        """
```

```
        Args:
            token_ids: [B, max_seq_len] 토큰 인덱스
        Returns:
            [B, embed_dim] L2 정규화된 텍스트 임베딩
        """
        x = self.token_embed(token_ids) + self.pos_embed

        # Causal mask 생성 (텍스트는 자기회귀적으로 처리)
        seq_len = token_ids.shape[1]
        causal_mask = torch.triu(
            torch.ones(seq_len, seq_len, device=x.device) * float('
            diagonal=1
        )

        x = self.transformer(x, mask=causal_mask)
        x = self.layer_norm(x)

        # [EOS] 토큰 위치의 출력을 사용 (가장 마지막 유효 토큰)
        # 여기서는 간략화를 위해 마지막 토큰 사용
        x = x[:, -1]  # [B, embed_dim]

        x = self.projection(x)
        x = F.normalize(x, dim=-1)

        return x


class CLIP(nn.Module):
    """CLIP 전체 모델: 이미지 인코더 + 텍스트 인코더 + Contrastive Los

    def __init__(self, embed_dim=768):
        super().__init__()
        self.image_encoder = CLIPImageEncoder(embed_dim=embed_dim)
        self.text_encoder = CLIPTextEncoder(embed_dim=embed_dim)

        # 학습 가능한 온도 파라미터 (log scale)
        # τ가 커지면 유사도 분포가 더 뾰족해짐
        self.log_temperature = nn.Parameter(torch.tensor(0.07).log(

    def forward(self, images, token_ids):
        """
        Args:
            images: [B, 3, 224, 224] 이미지 배치
            token_ids: [B, 77] 텍스트 토큰 배치
        Returns:
            loss: CLIP contrastive loss (스칼라)
        """
        # 각 인코더로 임베딩 생성
        z_img = self.image_encoder(images)    # [B, embed_dim]
        z_txt = self.text_encoder(token_ids)  # [B, embed_dim]
```

```
        # 온도 스케일링된 코사인 유사도 행렬 계산
        temperature = self.log_temperature.exp()
        # z_img, z_txt가 이미 L2 정규화되어 있으므로 내적 = 코사인 유사
        logits = (z_img @ z_txt.T) / temperature  # [B, B]

        # 대각선이 정답인 N-way 분류 문제
        labels = torch.arange(len(images), device=images.device)

        # Image→Text 방향 + Text→Image 방향 손실 평균
        loss_i2t = F.cross_entropy(logits, labels)        # 행 방향
        loss_t2i = F.cross_entropy(logits.T, labels)      # 열 방향
        loss = (loss_i2t + loss_t2i) / 2.0

        return loss

    @torch.no_grad()
    def get_image_features(self, images):
        """추론용: 이미지 임베딩 추출"""
        return self.image_encoder(images)

    @torch.no_grad()
    def get_text_features(self, token_ids):
        """추론용: 텍스트 임베딩 추출"""
        return self.text_encoder(token_ids)


# === 학습 루프 예시 ===
def train_clip():
    """CLIP 학습 루프 (간략화)"""
    model = CLIP(embed_dim=768).cuda()
    optimizer = torch.optim.AdamW(model.parameters(), lr=5e-4, weigl

    for epoch in range(num_epochs):
        for images, token_ids in dataloader:
            images = images.cuda()       # [B, 3, 224, 224]
            token_ids = token_ids.cuda()  # [B, 77]

            loss = model(images, token_ids)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            # 온도 파라미터 클램핑 (폭발 방지)
            with torch.no_grad():
                model.log_temperature.clamp_(max=torch.tensor(100.0
```

⚠️ **주의**: 실제 CLIP 학습에는 수백 개의 GPU에서 배치 크기 32,768로 학습한다. 위 코드는 구조 이해를 위한 간략화 버전이다.

> 💡 **코드에서 주목해야 할 핵심 설계 결정들:**

> 1. **`self.log_temperature = nn.Parameter(torch.tensor(0.07).log())`**: Temperature를 직접 학습하지 않고 **log scale**로 파라미터화한다. 이유는 (a) temperature는 항상 양수여야 하므로 $\tau = \exp(\log \tau)$로 자동 보장하고, (b) log space에서의 gradient가 더 안정적이기 때문이다. 초기값 0.07은 CLIP 논문에서 제안한 값으로, $\exp(\log 0.07) = 0.07$에서 시작하여 학습을 통해 적절한 값을 찾아간다.
> 2. **`F.normalize(x, dim=-1)`**: 이 한 줄이 코사인 유사도의 핵심이다. 벡터를 단위 벡터로 만들면, 이후 `z_img @ z_txt.T`라는 단순한 행렬 곱셈이 곧 코사인 유사도 행렬이 된다. 정규화 없이 dot product를 사용하면 벡터 크기에 의한 편향이 발생한다.
> 3. **`F.cross_entropy(logits, labels)`**: PyTorch의 `cross_entropy`는 내부적으로 softmax + negative log likelihood를 수행한다. `labels = torch.arange(N)`은 대각선 위치가 정답임을 나타낸다. 즉, i번째 이미지의 정답은 i번째 텍스트이다.
> 4. **`model.log_temperature.clamp_(max=torch.tensor(100.0).log())`**: Temperature가 지나치게 커지면 softmax가 거의 one-hot에 가까워져서 gradient가 사라지거나 수치적으로 불안정해진다. 상한을 두어 이를 방지한다.

1.5 CLIP의 놀라운 능력: Zero-Shot Classification

> 💡 **왜 Zero-Shot Classification이 놀라운가?** 전통적인 이미지 분류는 다음 과정을 거친다: (1) ImageNet의 1000개 클래스에 대해 레이블이 달린 128만 장의 이미지를 준비한다, (2) ResNet/ViT를 이 데이터로 수십 시간~수일 학습시킨다, (3) 테스트 이미지를 분류한다. 새로운 클래스(예: "코기 강아지")를 추가하려면? 그 클래스의 이미지를 모아서 **처음부터 다시 학습**해야 한다.
>
> CLIP은 이 전체 과정을 건너뛴다. ImageNet 이미지를 **단 한 장도 보지 않고** 76.2% 정확도를 달성한다. 이것이 가능한 근본적 이유는 CLIP이 "고양이 = 클래스 #281"이라는 임의의 매핑이 아니라, **"고양이라는 개념의 의미"** 자체를 학습했기 때문이다. 텍스트 인코더가 "a photo of a cat"이라는 문장에서 "고양이"의 시각적 의미를 이해하고, 이미지 인코더가 고양이 사진의 시각적 특성을 같은 공간에 매핑하므로, 둘이 자연스럽게 가까워진다.

CLIP은 분류기로 학습한 적이 없지만, ImageNet을 **zero-shot으로** 분류할 수 있다:

```
1. 모든 클래스 이름을 텍스트 프롬프트로 변환:
   "a photo of a {class_name}" × 1000 클래스

2. 텍스트 인코더로 1000개의 텍스트 임베딩 생성

3. 테스트 이미지의 이미지 임베딩과 1000개의 텍스트 임베딩의 유사도 계산

4. 가장 유사한 텍스트의 클래스를 예측으로 사용
```

결과: ImageNet top-1 정확도 **76.2%** (별도 학습 없이!)

> 💡 **왜 이것이 작동하는가? -- 단계별 직관**
>
> Zero-shot classification이 작동하는 이유를 깊이 이해해보자:
>
> 1. **CLIP은 공유 의미 공간(shared semantic space)을 학습했다**: 4억 개의 (이미지, 텍스트) 쌍으로 학습하면서, "dog"라는 단어가 포함된 캡션은 개 사진과 가까이, 고양이 사진과는 멀리 배치되었다. 학습이 끝나면, "dog"의 텍스트 임베딩 주변에는 실제 개 이미지의 임베딩들이 군집을 이루고 있다.
> 2. **텍스트 인코더가 "즉석 분류기(on-the-fly classifier)"를 생성한다**: "a photo of a dog"라는 텍스트를 인코딩하면, 이것은 곧 "개 이미지 군집의 중심점"에 해당하는 벡터가 된다. 1000개 클래스에 대해 이 작업을 수행하면, 1000개의 "클래스 중심점(class prototype)"이 만들어진다. 이것은 사실상 1000-class linear classifier의 가중치 행렬과 같은 역할을 한다!
> 3. **코사인 유사도가 분류 확률이 된다**: 테스트 이미지의 임베딩과 1000개 텍스트 임베딩의 유사도를 계산하면, 이것은 softmax classifier의 logit과 동일한 구조이다. argmax를 취하면 예측 클래스가 된다.
> 4. **Fine-tuning이 불필요한 이유**: 전통적 분류기는 학습 데이터에서 각 클래스의 "프로토타입"을 학습한다. CLIP에서는 텍스트 인코더가 자연어로부터 이 프로토타입을 **즉석에서 생성**한다. 따라서 "Pembroke Welsh Corgi"라는 처음 보는 세부 품종에 대해서도, 텍스트 인코더가 그 단어들의 의미를 조합해서 적절한 프로토타입을 만들어낸다.
>
> **Prompt engineering의 중요성**: "cat"만 입력하는 것보다 "a photo of a cat"이 더 좋은 이유는, CLIP이 학습한 캡션들이 대부분 "a photo of...", "an image of..." 형태였기 때문이다. 여러 템플릿의 평균을 사용하면 ("a photo of a cat", "a picture of a cat", "a cat in the wild" 등) 프로토타입이 더 robust해져서 성능이 올라간다.

```python
@torch.no_grad()
def zero_shot_classify(clip_model, image, class_names, templates=No
    """
    CLIP을 이용한 Zero-Shot 분류

    Args:
        clip_model: 학습된 CLIP 모델
        image: [1, 3, 224, 224] 분류할 이미지
        class_names: ['cat', 'dog', 'car', ...] 클래스 이름 리스트
        templates: 프롬프트 템플릿 리스트 (없으면 기본값 사용)
    Returns:
        predicted_class: 예측된 클래스 이름
    """
    if templates is None:
        templates = ["a photo of a {}."]

    # 1. 모든 클래스에 대해 텍스트 임베딩 생성
    text_features_list = []
    for class_name in class_names:
        # 여러 템플릿의 평균을 사용하면 성능이 더 좋음
        class_embeddings = []
        for template in templates:
            text = template.format(class_name)
            tokens = tokenize(text)  # 토크나이저로 변환
            feat = clip_model.get_text_features(tokens.cuda())
            class_embeddings.append(feat)
        # 템플릿들의 평균 임베딩
        class_embedding = torch.stack(class_embeddings).mean(dim=0)
        class_embedding = F.normalize(class_embedding, dim=-1)
        text_features_list.append(class_embedding)

    text_features = torch.cat(text_features_list, dim=0)  # [num_cl

    # 2. 이미지 임베딩 생성
    image_features = clip_model.get_image_features(image.cuda())  #

    # 3. 코사인 유사도 계산
    similarity = (image_features @ text_features.T).squeeze(0)  # [

    # 4. 가장 유사한 클래스 반환
    predicted_idx = similarity.argmax().item()
    return class_names[predicted_idx]
```

1.6 생성 모델에서 CLIP의 역할

CLIP 자체는 **생성 모델이 아니다**. 생성 모델의 **조건 인코더**로 사용된다:

> 💡 **왜 생성 모델이 CLIP을 필요로 하는가?** 디퓨전 모델은 본질적으로 **이미지 공간**에서 작동한다: 노이즈를 이미지로 변환하는 과정을 학습한다. 하지만 사용자의 조

> 건은 **텍스트 공간**에 있다. 이 두 공간을 연결하는 "번역기"가 필요한데, CLIP이 바로 그 역할을 한다. CLIP은 4억 개의 데이터로 "텍스트의 의미를 시각적 의미와 정렬하는 방법"을 이미 학습했으므로, 디퓨전 모델은 CLIP의 텍스트 인코더를 그대로 가져다 쓰면 된다. 처음부터 텍스트-이미지 관계를 학습할 필요가 없다 -- 이것이 **전이 학습(transfer learning)**의 힘이다.

```
"a cat sitting on a throne"
    ↓
[CLIP Text Encoder]
    ↓
텍스트 임베딩 c ∈ ℝ^{77×768}     ← 77 토큰, 각 768차원
    ↓
[Cross-Attention으로 U-Net/DiT에 주입]
    ↓
조건부 디노이징
```

생성 모델	CLIP 사용 방식
DALL-E 2	CLIP 이미지 임베딩 예측 → Diffusion Decoder
Stable Diffusion 1.x	CLIP ViT-L/14 텍스트 인코더
SDXL	CLIP ViT-L + OpenCLIP ViT-bigG 듀얼
SD 3	CLIP ViT-L + CLIP ViT-bigG + T5-XXL 트리플

왜 SD 3에서 T5를 추가하는가?

- CLIP: 이미지-텍스트 정합에 최적화 → 시각적 개념("빨간 모자")에 강함
- T5: 순수 언어 모델 → 복잡한 문장("왼쪽의 빨간 모자를 쓴 여자가 오른쪽의 파란 코트를 입은 남자를 바라본다")에 강함
- 두 종류를 결합하면 각각의 약점을 보완

> 💡 **CLIP의 텍스트 이해 한계와 T5의 보완** CLIP은 contrastive learning으로 학습되었기 때문에, "이미지와 매칭되는 짧은 캡션"에 최적화되어 있다. 하지만 자연어의 복잡한 구조 -- 부정문("고양이가 **없는** 방"), 수량("**세** 마리의 고양이"), 공간 관계("고양이가 개 **위에**") -- 를 정확히 파싱하는 능력은 부족하다. T5 같은 대규모 언어 모델은 텍스트-텍스트 과제로 사전학습되어 이런 언어적 뉘앙스를 훨씬 잘 이해한다. SD3에서 두 인코더의 출력을 결합(concatenate)하여 Cross-Attention에 주입하면, 시각적 개념은 CLIP이, 복잡한 언어 구조는 T5가 담당하는 분업이 이루어진다.

⚠️ **참고**: Stable Diffusion에서 사용하는 CLIP Text Encoder는 [CLS]/[EOS] 토큰의 단일 벡터가 아니라, **모든 토큰의 시퀀스 출력** $c \in \mathbb{R}^{77\times768}$을 사용한다. 이를 통해 Cross-Attention에서 토큰별 세밀한 조건 주입이 가능하다.

> 💡 **[CLS] 벡터 vs 시퀀스 출력 -- 왜 시퀀스 전체를 사용하는가?** Zero-shot classification에서는 [EOS] 토큰의 단일 벡터 $z \in \mathbb{R}^{768}$만 사용했다. 이것은 "전체 텍스트의 요약"이므로 분류에는 충분하다. 하지만 이미지 **생성**에서는 "빨간 자동

> 차가 초록 잔디 위에 있다"라는 문장에서 "빨간"이 어디에, "초록"이 어디에 적용되어야 하는지 토큰별로 구분해야 한다. 단일 벡터에는 이런 세밀한 정보가 압축되어 손실된다. 시퀀스 출력 $c \in \mathbb{R}^{77\times768}$을 사용하면, Cross-Attention에서 이미지의 각 공간 위치가 텍스트의 개별 토큰에 선택적으로 주목(attend)할 수 있다.

---

2. 조건부 디퓨전: 텍스트를 이미지 생성에 주입하는 방법

2.1 Cross-Attention — 표준 방법

> 💡 **왜 Cross-Attention이 텍스트 조건 주입에 적합한가?** 텍스트 조건을 이미지 생성에 주입하는 방법은 여러 가지가 있다:
>
> **(1) 단순 결합(concatenation)**: 텍스트 벡터를 이미지 특징에 채널 차원으로 붙이는 방법. 문제: 텍스트의 모든 정보가 이미지의 **모든 위치**에 동일하게 전달된다. "red car on green grass"에서 "red"는 자동차 영역에만, "green"은 잔디 영역에만 영향을 줘야 하는데, concatenation으로는 이런 공간적 선택성을 구현할 수 없다.
>
> **(2) 덧셈(addition)**: 텍스트 벡터를 이미지 특징에 더하는 방법. concatenation과 같은 문제를 가진다 -- 모든 공간 위치에 동일한 조건이 더해진다.
>
> **(3) Cross-Attention**: 이미지의 **각 공간 위치가 독립적으로** 텍스트의 어떤 토큰에 주목할지 결정한다. 자동차 영역의 픽셀은 "red"와 "car" 토큰에 높은 attention weight를 줄 수 있고, 잔디 영역의 픽셀은 "green"과 "grass" 토큰에 높은 weight를 줄 수 있다. 이것이 **동적 공간 선택성(dynamic spatial selectivity)**이며, Cross-Attention이 조건 주입의 표준 방법이 된 핵심 이유이다.

강의 5의 DALL-E에서 배운 Cross-Attention과 동일한 메커니즘:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V$$

변수	출처	차원
$Q = W_Q \cdot \phi(z_t)$	이미지 잠재 특징	$[\text{pixels}, d]$
$K = W_K \cdot \tau(c)$	**텍스트 인코더 출력**	$[77, d]$
$V = W_V \cdot \tau(c)$	**텍스트 인코더 출력**	$[77, d]$

> 💡 **Q, K, V의 직관적 의미**
>
> Attention 메커니즘에서 Query, Key, Value는 **정보 검색(information retrieval)**의 비유로 이해하면 명쾌하다:
>
> - **Query (Q)** = "내가 찾고 있는 것은 무엇인가?" -- 이미지의 각 공간 위치가 "나는 어떤 텍스트 정보를 필요로 하는가?"라고 질문한다. 예를 들어, 자동

차를 생성하고 있는 위치의 Query는 "나의 색상은 무엇이어야 하는가?"라는 질문을 암시적으로 인코딩한다.

- **Key (K)** = "내가 담고 있는 것은 무엇인가?" -- 텍스트의 각 토큰이 "나는 이런 정보를 가지고 있다"고 광고한다. "red" 토큰의 Key는 "나는 색상 정보를 가지고 있다"를, "car" 토큰의 Key는 "나는 객체 정보를 가지고 있다"를 인코딩한다.
- **Value (V)** = "내가 제공하는 실제 정보" -- Key가 "광고"라면, Value는 실제로 전달되는 "상품"이다. attention weight에 따라 가중 합산되어 최종적으로 이미지 특징에 주입되는 정보이다.

$QK^\top$는 모든 Query-Key 쌍의 호환성(compatibility)을 계산한다. 이것에 softmax를 적용하면 "이미지의 각 위치가 텍스트의 각 토큰에 얼마나 주목하는가"를 나타내는 attention weight가 된다. 이 weight로 Value를 가중 합산하면, 각 이미지 위치에 적절한 텍스트 정보가 주입된다.

💡 **$\frac{1}{\sqrt{d}}$ 스케일링은 왜 필요한가?** Q와 K의 각 원소가 평균 0, 분산 1인 독립적인 값이라면, d차원 벡터의 내적 $q \cdot k = \sum_{i=1}^{d} q_i k_i$의 분산은 d가 된다 (각 항의 분산이 1이므로 d개를 합산). d가 64나 128처럼 크면 내적 값이 매우 커져서 softmax가 거의 one-hot에 가까워지고, gradient가 매우 작아져 학습이 어려워진다. $\frac{1}{\sqrt{d}}$로 나누면 내적의 분산이 1로 정규화되어 softmax가 적절한 범위에서 작동한다.

- $\phi(z_t)$: U-Net/DiT의 중간 특징 맵을 평탄화한 것
- $\tau(c)$: CLIP/T5 텍스트 인코더의 토큰별 출력 (77개 토큰 × 768차원)

Self-Attention: Q, K, V 모두 이미지에서 → 이미지 내부의 관계

Cross-Attention: Q는 이미지, K/V는 텍스트 → 텍스트 조건을 이미지에 주입

Attention 계산의 차원 추적 (Step-by-step)

구체적인 숫자로 Cross-Attention의 차원 흐름을 추적해보자. 32x32 해상도의 특징 맵에서 8-head attention을 사용한다고 가정하자:

단계	연산	Shape	설명
입력 (이미지)	$\phi(z_t)$	$[B, 1024, 512]$	32x32=1024 위치, 512 채널
입력 (텍스트)	$\tau(c)$	$[B, 77, 768]$	77 토큰, CLIP 768차원
Q 계산	$W_Q \cdot \phi(z_t)$	$[B, 1024, 512]$	이미지에서 Query 생성
K 계산	$W_K \cdot \tau(c)$	$[B, 77, 512]$	텍스트에서 Key 생성
V 계산	$W_V \cdot \tau(c)$	$[B, 77, 512]$	텍스트에서 Value 생성

멀티헤드 분할	reshape	Q: $[B, 8, 1024, 64]$, K: $[B, 8, 77, 64]$	8헤드, 각 64차원
Attention score	$QK^\top/\sqrt{64}$	$[B, 8, 1024, 77]$	각 이미지 위치가 각 텍스트 토큰에 대한 점수
Attention weight	softmax(score)	$[B, 8, 1024, 77]$	77개 토큰에 대한 확률 분포
출력	weight × V	$[B, 8, 1024, 64]$	텍스트 정보가 주입된 이미지 특징
헤드 병합	concat + proj	$[B, 1024, 512]$	원래 이미지 특징과 같은 shape

핵심: attention weight의 shape이 $[B, 8, 1024, 77]$이라는 것은, **이미지의 1024개 위치 각각이 텍스트의 77개 토큰에 대해 독립적인 attention 분포를 가진다**는 뜻이다. 이것이 공간 선택적 조건 주입의 핵심이다.

U-Net에서의 배치:

```
각 해상도 레벨에서:
  h → [Self-Attention]  → h'  (이미지 내부 관계)
  h'→ [Cross-Attention] → h'' (텍스트 조건 주입)
  h''→ [Feed-Forward]   → h'''
```

2.2 PyTorch 구현: Cross-Attention Layer

```python
class CrossAttention(nn.Module):
    """
    텍스트 조건을 이미지 특징에 주입하는 Cross-Attention 레이어.

    Q: 이미지 잠재 특징에서 생성 → "이미지가 텍스트에게 질문"
    K, V: 텍스트 인코더 출력에서 생성 → "텍스트가 답변을 제공"
    """

    def __init__(self, query_dim, context_dim=768, num_heads=8, hea
        """
        Args:
            query_dim: 이미지 특징의 차원 (U-Net 채널 수)
            context_dim: 텍스트 인코더 출력 차원 (CLIP=768)
            num_heads: 어텐션 헤드 수
            head_dim: 각 헤드의 차원
        """
        super().__init__()
        inner_dim = num_heads * head_dim
        self.num_heads = num_heads
        self.head_dim = head_dim
        self.scale = head_dim ** -0.5  # 1/sqrt(d_k)

        # Q는 이미지 특징에서, K/V는 텍스트에서 생성
        self.to_q = nn.Linear(query_dim, inner_dim, bias=False)
        self.to_k = nn.Linear(context_dim, inner_dim, bias=False)
        self.to_v = nn.Linear(context_dim, inner_dim, bias=False)

        # 출력 프로젝션
        self.to_out = nn.Linear(inner_dim, query_dim)

    def forward(self, x, context):
        """
        Args:
            x: [B, H*W, query_dim] 이미지 잠재 특징 (평탄화된 공간 차원
            context: [B, 77, context_dim] CLIP 텍스트 인코더 출력
        Returns:
            [B, H*W, query_dim] 텍스트 조건이 주입된 이미지 특징
        """
        B, N, _ = x.shape  # N = H*W (이미지 토큰 수)
        S = context.shape[1]  # S = 77 (텍스트 토큰 수)
        H = self.num_heads

        # Q, K, V 계산 후 멀티헤드 형태로 변환
        q = self.to_q(x).view(B, N, H, self.head_dim).transpose(1, 
        k = self.to_k(context).view(B, S, H, self.head_dim).transpos
        v = self.to_v(context).view(B, S, H, self.head_dim).transpos

        # 어텐션 스코어 계산: 이미지의 각 위치가 텍스트의 어떤 토큰에 주
        attn_weights = (q @ k.transpose(-2, -1)) * self.scale  # [B,
        attn_weights = F.softmax(attn_weights, dim=-1)

        # 어텐션 적용: 텍스트 정보를 이미지 각 위치에 주입
```

```python
        out = attn_weights @ v  # [B, H, N, d]

        # 헤드 병합 후 출력 프로젝션
        out = out.transpose(1, 2).contiguous().view(B, N, -1)  # [B,
        out = self.to_out(out)  # [B, N, query_dim]

        return out


class TransformerBlock(nn.Module):
    """
    U-Net 내부의 Transformer 블록.
    Self-Attention → Cross-Attention → Feed-Forward 순서로 구성.
    """

    def __init__(self, dim, context_dim=768, num_heads=8):
        super().__init__()

        # Self-Attention: 이미지 내부의 관계 학습
        self.norm1 = nn.LayerNorm(dim)
        self.self_attn = CrossAttention(
            query_dim=dim, context_dim=dim, num_heads=num_heads
        )

        # Cross-Attention: 텍스트 조건 주입
        self.norm2 = nn.LayerNorm(dim)
        self.cross_attn = CrossAttention(
            query_dim=dim, context_dim=context_dim, num_heads=num_he
        )

        # Feed-Forward Network
        self.norm3 = nn.LayerNorm(dim)
        self.ff = nn.Sequential(
            nn.Linear(dim, dim * 4),
            nn.GELU(),
            nn.Linear(dim * 4, dim),
        )

    def forward(self, x, context):
        """
        Args:
            x: [B, H*W, dim] 이미지 특징
            context: [B, 77, context_dim] 텍스트 조건 임베딩
        """
        # Self-Attention (이미지 ↔ 이미지)
        x = x + self.self_attn(self.norm1(x), context=self.norm1(x))

        # Cross-Attention (이미지 ← 텍스트)
        x = x + self.cross_attn(self.norm2(x), context=context)

        # Feed-Forward
        x = x + self.ff(self.norm3(x))
```

```
        x = x + self.ff(self.norm3(x))

        return x
```

2.3 AdaLN — DiT에서의 조건 주입

DiT(Diffusion Transformer)에서는 Cross-Attention 대신 **Adaptive Layer Normalization**을 사용한다:

$$\mathrm{AdaLN}(h, c) = \gamma(c) \odot \mathrm{LayerNorm}(h) + \beta(c)$$

- $\gamma(c), \beta(c)$: 조건 벡터 c에서 MLP로 생성한 스케일/시프트 파라미터
- 각 블록의 정규화 단계에서 조건을 주입

> 💡 **AdaLN의 직관: "채널별 볼륨 조절"** 일반 LayerNorm은 특징을 정규화한 뒤 고정된 γ, β로 스케일/시프트한다. AdaLN은 이 γ, β를 **조건 c에 따라 동적으로 생성**한다. 직관적으로, 조건이 "밤 풍경"이면 어두운 채널의 γ를 키우고 밝은 채널의 γ를 줄이는 식이다. 이것은 마치 이퀄라이저의 주파수별 볼륨을 조절하듯, 조건에 따라 각 특징 채널의 "중요도"를 조절하는 것이다.
>
> Cross-Attention과의 핵심 차이: AdaLN은 조건 벡터 하나가 **모든 공간 위치에 동일하게** 적용된다 (글로벌 조건 주입). 따라서 클래스 레이블처럼 전체 이미지에 균일하게 적용되는 조건에 적합하다. 반면, "왼쪽에 고양이, 오른쪽에 개"처럼 공간적으로 다른 조건이 필요한 경우에는 Cross-Attention이 필수적이다.

> Cross-Attention vs AdaLN: 전자는 토큰별 세밀한 조건 주입, 후자는 글로벌 조건 주입. SD3의 MM-DiT는 둘을 결합한다.

---

3. Classifier Guidance — 분류기를 이용한 안내

3.1 아이디어

> 📄 Dhariwal & Nichol, "Diffusion Models Beat GANs on Image Synthesis", NeurIPS 2021

> 💡 **왜 "Guidance(안내)"가 필요한가?** 무조건부 디퓨전 모델 $p(x)$는 학습 데이터의 전체 분포를 학습한다. 따라서 샘플링하면 개, 고양이, 풍경, 자동차 등이 무작위로 나온다. 이것은 **다양하지만 초점이 없다(diverse but unfocused)**. 사용자가 "개"를 원한다면, 생성 과정을 $p(x \mid y = \mathrm{dog})$ 방향으로 **유도(guide)**해야 한다. Guidance는 이 유도의 메커니즘이다.
>
> 비유하자면, 무조건부 모델은 "아무 음악이나 연주하는 오케스트라"이고, guidance는 "이번에는 베토벤 5번을 연주해주세요"라는 지시를 주는 것이다.

무조건부 디퓨전 모델이 있을 때, 별도의 분류기 $p_\phi(y \mid x_t)$의 기울기를 이용하여 특정 클래스로 유도한다.

베이즈 정리에서 출발하는 유도:

$$\nabla_{x_t} \log p(x_t \mid y) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y \mid x_t)$$

이 등식이 어떻게 나오는지 단계별로 유도해보자:

$$p(x_t \mid y) = \frac{p(y \mid x_t) \cdot p(x_t)}{p(y)} \quad \text{(Bayes' rule)}$$

양변에 log를 취하면:

$$\log p(x_t \mid y) = \log p(y \mid x_t) + \log p(x_t) - \log p(y)$$

x_t에 대해 gradient ∇_{x_t}를 취하면 ($p(y)$는 x_t에 의존하지 않으므로 소멸):

$$\nabla_{x_t} \log p(x_t \mid y) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(y \mid x_t)$$

이 결과의 의미: **조건부 스코어** = **무조건부 스코어** + **분류기 gradient**. 즉, 우리가 원하는 조건부 생성의 방향은, 이미 알고 있는 무조건부 방향에 분류기가 알려주는 "클래스 방향"을 더하면 된다.

항	의미	출처
$\nabla_{x_t} \log p(x_t)$	무조건부 스코어	사전 학습된 디퓨전 모델
$\nabla_{x_t} \log p(y \mid x_t)$	"클래스 y의 방향"	**별도로 학습한 분류기**

스케일 파라미터 s로 분류기의 영향을 조절:

$$\hat{\epsilon}(x_t, t, y) = \epsilon_\theta(x_t, t) - s \cdot \sqrt{1 - \bar{\alpha}_t} \cdot \nabla_{x_t} \log p_\phi(y \mid x_t)$$

> 💡 **스케일 s의 역할: 왜 $s > 1$이 작동하는가?** $s = 1$이면 정확한 베이즈 후분포 $p(x_t \mid y)$에서 샘플링하는 것이다. $s > 1$은 분류기의 gradient를 **증폭**한다. 이것은 실제 후분포보다 **더 뾰족한(sharpened)** 분포에서 샘플링하는 것과 같다: $p(x_t \mid y)^s$에 비례하는 분포. s가 커질수록 "확실히 그 클래스처럼 보이는" 이미지 쪽으로 더 강하게 편향된다. 마치 사진의 대비(contrast)를 높이는 것과 같은 효과이다.

3.2 한계

- **별도의 분류기**를 노이즈 이미지에서 학습해야 함 (시간별 분류기)
- 분류기의 기울기가 부정확하면 생성 품질 저하
- 텍스트 같은 연속적 조건에 적용하기 어려움

> 💡 **이 한계들이 왜 심각한가?**
>
> **(1) 별도 분류기 학습의 비용**: 일반 이미지 분류기와 달리, 여기서 필요한 분류기는 **노이즈가 섞인 이미지** x_t에서 작동해야 한다. 그리고 노이즈 레벨은 t에 따라 달라

> 지므로, 사실상 모든 t에서 작동하는 분류기를 학습해야 한다. 이것은 디퓨전 모델 자체를 학습하는 것만큼 비용이 클 수 있다.
>
> **(2) 범용성 부족**: 분류기는 사전에 정의된 클래스(예: ImageNet 1000 클래스)에 대해서만 작동한다. "a cat wearing a top hat in the style of Monet"같은 자유 텍스트 조건에는 분류기를 만들 수 없다. 즉, classifier guidance는 본질적으로 **닫힌 어휘 (closed vocabulary)** 시스템이다.
>
> **(3) Gradient 품질 문제**: 분류기의 gradient $\nabla_{x_t} \log p_\phi(y \mid x_t)$가 adversarial attack처럼 작동할 수 있다 -- 인간이 보기에는 의미 없는 패턴이지만 분류기가 높은 확률을 부여하는 방향으로 이미지를 변형시킬 수 있다. 이는 이미지 품질을 저하시킨다.

이 한계를 극복한 것이 Classifier-Free Guidance이다.

---

4. Classifier-Free Guidance (CFG)

> 📄 Ho & Salimans, "Classifier-Free Diffusion Guidance", 2022

4.1 핵심 아이디어

> 💡 **왜 Classifier-Free인가? -- Classifier Guidance의 문제를 되짚어보자** Classifier Guidance는 $\nabla_{x_t} \log p(c \mid x_t)$를 계산하기 위해 별도의 분류기를 학습해야 했다. CFG의 핵심 통찰은 이것이다: **분류기의 gradient를 직접 계산하지 않고도, 같은 효과를 낼 수 있다.** 어떻게? 조건부 모델과 무조건부 모델의 차이가 곧 분류기 gradient와 동일한 정보를 담고 있다는 수학적 사실을 이용하는 것이다. 그리고 "조건부 모델"과 "무조건부 모델"을 별도로 만들 필요도 없다 -- **하나의 모델**이 조건을 받을 때와 받지 않을 때 다르게 행동하도록 학습시키면 된다.

별도의 분류기 없이, **동일한 디퓨전 모델** 하나로 조건부와 무조건부를 모두 학습한다.

⚠️ **핵심 통찰**: 모델 하나가 조건부 예측과 무조건부 예측을 모두 할 수 있다면, 두 예측의 **차이**가 곧 "조건의 방향"이 된다. 별도의 분류기가 필요 없다!

이것은 놀라울 정도로 단순하면서도 강력한 아이디어이다. Classifier Guidance가 "전문가(분류기)를 따로 고용"하는 것이라면, CFG는 "같은 사람이 두 가지 역할을 번갈아 하면서, 그 차이에서 방향을 찾는 것"이다.

4.2 학습: 조건 드롭아웃

> 💡 **왜 "드롭아웃"이라는 이름인가?** 일반적인 dropout은 뉴런의 활성화를 랜덤하게 0으로 만들어서 과적합을 방지하는 정규화 기법이다. 조건 드롭아웃(condition dropout)은 이와 유사하게, **조건 입력 자체를 랜덤하게 "끈다"**. 하지만 목적은 정규화가 아니라, 하나의 모델이 두 가지 모드로 작동하게 만드는 것이다: 조건이 있

> 을 때(조건부 모드)와 없을 때(무조건부 모드). 이렇게 하면 추론 시 두 모드의 출력을 비교할 수 있다.

학습 시 일정 확률(보통 10~20%)로 조건 c를 **빈 조건** $\varnothing$로 교체:

$$\text{학습 입력} = \begin{cases} (x_t, t, c) & \text{확률 } 1 - p_{\text{uncond}} \\ (x_t, t, \varnothing) & \text{확률 } p_{\text{uncond}} \end{cases}$$

- $\varnothing$: 빈 문자열의 CLIP 임베딩, 또는 학습된 null 토큰
- $p_{\text{uncond}} = 0.1$ (일반적)

> 같은 모델이 조건부($\epsilon_\theta(x_t, t, c)$)와 무조건부($\epsilon_\theta(x_t, t, \varnothing)$) 역할을 모두 수행.

> 💡 **p_{uncond}의 선택이 중요한 이유** p_{uncond}가 너무 크면(예: 0.5) 학습 데이터의 절반에서 조건 정보를 버리므로, 모델의 조건부 생성 능력이 약해진다. 너무 작으면(예: 0.01) 무조건부 예측 능력이 충분히 학습되지 않아, 추론 시 $\epsilon_\theta(x_t, t, \varnothing)$의 품질이 떨어진다. 0.1~0.2가 경험적으로 좋은 균형점이며, 이는 모델 용량의 약 10~20%를 무조건부 학습에 할당하는 것으로 해석할 수 있다.

4.3 생성: 가중 외삽

$$\boxed{\tilde{\epsilon}_\theta(x_t, t, c) = \epsilon_\theta(x_t, t, \varnothing) + w \cdot (\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing))}$$

정리하면:

$$\tilde{\epsilon} = (1 - w) \cdot \epsilon_\theta(x_t, t, \varnothing) + w \cdot \epsilon_\theta(x_t, t, c)$$

> 💡 **이 수식이 의미하는 것을 직관적으로 이해하기**
>
> 이 수식을 **벡터 연산**으로 이해하면 명쾌하다. $\epsilon_\theta(x_t, t, \varnothing)$과 $\epsilon_\theta(x_t, t, c)$는 모두 이미지와 같은 shape의 텐서(벡터)이다.
>
> - $\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)$는 **"조건 방향 벡터"**이다. "무조건부에서 조건부로 가는 방향"을 나타낸다. 이것이 곧 "조건 c가 노이즈 예측에 미치는 순수한 영향"이다.
> - w는 이 방향 벡터의 **크기를 제어**한다. $w = 1$이면 조건부 예측을 그대로 사용하는 것이고, $w > 1$이면 조건의 영향을 **과장(amplify)**하는 것이다.
> - $w > 1$일 때 이것은 **외삽(extrapolation)**이다. 보간(interpolation, $0 \leq w \leq 1$)은 두 점 사이를 이동하지만, 외삽($w > 1$)은 그 방향으로 더 멀리 나아간다. 마치 사진의 대비(contrast)를 높이는 것처럼, 조건에 맞는 특성은 더 강화되고 맞지 않는 특성은 더 억제된다.

w	효과	이름
$w = 0$	무조건부 생성	조건 무시
$w = 1$	표준 조건부 생성	보통 생성

$w = 3 \sim 7$	**조건에 더 충실** (일반적 사용값)	Guided generation
$w = 15+$	과도한 가이던스 → 이미지 품질 저하, 포화	Over-guidance

> 💡 **왜 $w > 1$이 놀라울 정도로 잘 작동하는가?** 수학적으로 $w = 1$이 "정확한" 조건부 분포 $p(x \mid c)$이다. $w > 1$은 이 분포를 넘어선 것이므로, 이론적으로는 분포 밖의 샘플을 생성하는 셈이다. 그런데 왜 결과가 더 좋은가? 이유는 두 가지이다:
>
> 1. **모델의 불완전성 보상**: 실제 모델은 $p(x \mid c)$를 완벽하게 학습하지 못한다. 조건 c의 영향이 학습 과정에서 과소평가(underfitting)되는 경향이 있다. $w > 1$은 이 과소평가를 보상하여, 실제 의도에 더 가까운 결과를 만든다.
> 2. **분포의 날카로움(sharpness) 증가**: $w > 1$은 실질적으로 $p(x \mid c)^w / Z$에서 샘플링하는 것과 비슷한 효과이다. 이것은 분포의 모드(mode) 근처에 확률이 집중되므로, "가장 대표적인" 샘플이 생성된다. 다양성은 줄지만 품질과 조건 충실도는 올라간다.

4.4 수학적 해석

> 💡 **이 절에서는 CFG가 Classifier Guidance와 수학적으로 동치임을 단계별로 증명한다.**

CFG를 스코어 함수 관점에서 보면:

$$\tilde{s}(x_t, c) = s(x_t) + w \cdot (s(x_t, c) - s(x_t))$$
$$= s(x_t) + w \cdot \nabla_{x_t} \log p(c \mid x_t)$$

이것은 Classifier Guidance에서 $s \cdot \nabla \log p(y \mid x_t)$와 같은 형태이다.

CFG 공식의 Bayes' rule 유도 (Step-by-step)

이 동치를 처음부터 유도해보자.

Step 1: Score function과 노이즈 예측의 관계

디퓨전 모델에서 스코어 함수(score function)와 노이즈 예측은 다음 관계를 가진다:

$$\nabla_{x_t} \log p(x_t) = -\frac{1}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t)$$

직관: 스코어 함수 $\nabla_{x_t} \log p(x_t)$는 "x_t에서 데이터 밀도가 높아지는 방향"을 가리킨다. 노이즈 예측 ϵ_θ는 "x_t에 더해진 노이즈의 방향"을 가리킨다. 데이터 방향은 노이즈의 반대 방향이므로, 부호가 반대이다.

Step 2: 조건부와 무조건부 스코어

조건부 스코어: $\nabla_{x_t} \log p(x_t \mid c) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t, c)$

무조건부 스코어: $\nabla_{x_t} \log p(x_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t, \varnothing)$

Step 3: Bayes' rule에서 분류기 gradient 추출

Section 3에서 유도한 것처럼:

$$\nabla_{x_t} \log p(x_t \mid c) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(c \mid x_t)$$

따라서 "암시적 분류기 gradient"는:

$$\nabla_{x_t} \log p(c \mid x_t) = \nabla_{x_t} \log p(x_t \mid c) - \nabla_{x_t} \log p(x_t)$$

Step 4: 노이즈 예측으로 변환

위에 Step 1의 관계를 대입하면:

$$\nabla_{x_t} \log p(c \mid x_t) = -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \left(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\right)$$

이것이 핵심이다: **두 노이즈 예측의 차이가 곧 분류기의 gradient이다!** 별도의 분류기를 학습하지 않아도, 조건부 예측에서 무조건부 예측을 빼면 "조건 c가 x_t에 얼마나 부합하는지"의 방향을 알 수 있다.

Step 5: Guidance scale w를 적용하여 CFG 공식 완성

Classifier Guidance에서 스케일 s를 곱했던 것처럼, CFG에서도 이 암시적 분류기 gradient에 w를 곱한다:

$$\tilde{s}(x_t, c) = \nabla_{x_t} \log p(x_t) + w \cdot \nabla_{x_t} \log p(c \mid x_t)$$

이것을 노이즈 예측으로 변환하면:

$$\tilde{\epsilon} = \epsilon_\theta(x_t, t, \varnothing) + w \cdot \left(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\right)$$

이것이 바로 Section 4.3의 CFG 공식이다. 즉, CFG는 **Classifier Guidance의 분류기를 "조건부 - 무조건부" 차이로 대체한 것**이며, 수학적으로 완벽히 동일한 프레임워크이다.

다만 $\nabla \log p(c \mid x_t)$를 **암시적으로** 두 예측의 차이로 추정한다:

$$\nabla_{x_t} \log p(c \mid x_t) \approx -\frac{1}{\sqrt{1-\bar{\alpha}_t}} \left(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\right)$$

> **별도 분류기 불필요**. 같은 모델에서 조건부/무조건부 예측을 빼기만 하면 "분류기 기울기"를 얻는다.

> 💡 **CFG가 Classifier Guidance보다 우수한 근본적 이유** Classifier Guidance의 분류기 $p_\phi(c \mid x_t)$는 디퓨전 모델과 **별도로** 학습된다. 따라서 분류기가 보는 "노이즈 이미지의 특성"과 디퓨전 모델이 보는 "특성"이 다를 수 있다 (distribution mismatch). 반면 CFG의 "암시적 분류기"는 디퓨전 모델 자체에서 나오므로, 모델

> 이 학습한 표현과 완벽하게 정합(aligned)된다. 또한, 텍스트 같은 연속적이고 고차원적인 조건에 대해서도 자연스럽게 작동한다 -- 별도의 "텍스트 분류기"를 학습할 필요가 없다.

4.5 기하학적 직관

```
                          w=1 (표준)
                         ↗
무조건부 예측 ──────────→ 조건부 예측 ──────────→ w=7.5 (강한 가이던스)
ε(x_t, ∅)                ε(x_t, c)                ε̃(x_t, c)

"조건 없이 만든 이미지"  "조건 있는 이미지"      "조건을 과장한 이미지"
(아무 이미지)            (고양이)                (매우 선명한 고양이)
```

$w > 1$에서 **외삽(extrapolation)**이 일어난다:

- 무조건부 → 조건부 방향을 w배로 증폭
- 조건에 맞는 특징이 과장되면서 이미지 품질과 조건 충실도가 올라감
- 대신 다양성이 줄어듦 (품질-다양성 trade-off)

> 💡 **기하학적 직관을 더 깊이 이해하기**
>
> 노이즈 예측 ϵ은 이미지와 같은 차원의 고차원 벡터이다 (예: $64 \times 64 \times 4 = 16384$ 차원). 이 고차원 공간에서 일어나는 일을 2D로 단순화해서 시각화해보자:
>
> ```
> ε(x_t, c)  "고양이 방향"
> ↗
> /
> /        ε˜(w=7.5)
> /        ↗ 조건 방향으로 7.5배 외삽
> /        /
> ε(x_t, ∅)  ──────────→──────→
> "평균적 이미지 방향"
>
> ε˜= ε(∅) + 7.5 × [ε(c) - ε(∅)]
> = ε(∅) + 7.5 × (조건 방향 벡터)
> ```
>
> **왜 over-guidance ($w \geq 15$)에서 품질이 떨어지는가?** 외삽을 너무 많이 하면 벡터가 학습 데이터 분포의 support(지지) 밖으로 나간다. 이미지 공간에서 이것은 색상 포화(saturation), 부자연스러운 패턴, 아티팩트(artifact)로 나타난다. 마치 사진 편집에서 대비를 극단적으로 높이면 자연스러운 색감이 사라지는 것과 같다. 또한, 외삽된 벡터가 너무 크면 디노이징의 다음 스텝에서 x_{t-1}이 정상 범위를 벗어나, 연쇄적으로 품질이 악화된다.
>
> **품질-다양성 trade-off의 수학적 해석**: w가 커질수록 생성 분포가 $p(x \mid c)$의 모드(mode) 근처로 집중된다. 극단적으로 $w \to \infty$이면 모든 샘플이 동일한 "가장

> 대표적인 이미지"로 수렴한다. 반대로 $w = 0$이면 조건을 무시하고 $p(x)$에서 샘플링하므로 다양하지만 조건과 무관한 이미지가 나온다.

4.6 PyTorch 구현: CFG Sampling

아래는 CFG를 적용한 DDPM sampling 과정의 전체 구현이다:

```python
import torch
import torch.nn.functional as F


class CFGSampler:
    """
    Classifier-Free Guidance를 적용한 DDPM 샘플러.

    핵심: 매 디노이징 스텝마다 모델을 2번 호출한다.
      1) 무조건부 예측: ε_θ(x_t, t, ∅)
      2) 조건부 예측:   ε_θ(x_t, t, c)
    그리고 두 예측을 guidance scale w로 외삽한다.
    """

    def __init__(self, model, num_timesteps=1000, beta_start=1e-4, l
        """
        Args:
            model: 조건부 디퓨전 모델 ε_θ(x_t, t, c)
            num_timesteps: 총 디퓨전 스텝 수 T
            beta_start, beta_end: 노이즈 스케줄 범위
        """
        self.model = model
        self.T = num_timesteps

        # 노이즈 스케줄 설정
        betas = torch.linspace(beta_start, beta_end, num_timesteps)
        alphas = 1.0 - betas
        alphas_cumprod = torch.cumprod(alphas, dim=0)

        # 사전 계산 값 저장
        self.betas = betas
        self.alphas = alphas
        self.alphas_cumprod = alphas_cumprod
        self.sqrt_alphas = torch.sqrt(alphas)
        self.sqrt_one_minus_alphas_cumprod = torch.sqrt(1.0 - alpha
        self.sqrt_recip_alphas = 1.0 / torch.sqrt(alphas)

    @torch.no_grad()
    def sample(self, text_embedding, guidance_scale=7.5,
               image_shape=(1, 4, 64, 64), device='cuda'):
        """
        CFG를 적용한 전체 샘플링 과정.

        Args:
            text_embedding: [B, 77, 768] CLIP 텍스트 인코더 출력 (조건
            guidance_scale: w 값 (7.5가 일반적)
            image_shape: 생성할 이미지(또는 잠재 벡터)의 shape
            device: 연산 디바이스
        Returns:
            x_0: [B, C, H, W] 생성된 이미지(또는 잠재 벡터)
        """
        B = text_embedding.shape[0]
```

```
        # 빈 조건 임베딩 생성 (빈 문자열 ""의 CLIP 임베딩)
        # 실제로는 tokenizer("")를 CLIP에 통과시켜 얻음
        null_embedding = self._get_null_embedding(B, device)  # [B,

        # 1. 순수 가우시안 노이즈에서 시작: x_T ~ N(0, I)
        x_t = torch.randn(image_shape, device=device)

        # 2. 역방향 과정: t = T, T-1, ..., 1
        for t in reversed(range(self.T)):
            t_batch = torch.full((B,), t, device=device, dtype=torch

            # ========== CFG의 핵심: 2번의 forward pass ==========

            # (a) 무조건부 예측: 조건 없이 노이즈 예측
            eps_uncond = self.model(x_t, t_batch, null_embedding)

            # (b) 조건부 예측: 텍스트 조건으로 노이즈 예측
            eps_cond = self.model(x_t, t_batch, text_embedding)

            # (c) CFG 외삽: 무조건부에서 조건부 방향으로 w배 증폭
            eps_guided = eps_uncond + guidance_scale * (eps_cond - e

            # ==================================================

            # 3. DDPM 디노이징 스텝
            x_t = self._denoise_step(x_t, eps_guided, t)

        return x_t  # 최종 생성 결과 x_0

    def _denoise_step(self, x_t, predicted_noise, t):
        """
        DDPM의 단일 디노이징 스텝.

        x_{t-1} = (1/√α_t) * (x_t - (β_t/√(1-ᾱ_t)) * ε̂) + σ_t * z
        """
        alpha_t = self.alphas[t]
        alpha_bar_t = self.alphas_cumprod[t]
        beta_t = self.betas[t]

        # 평균 계산
        mean = self.sqrt_recip_alphas[t] * (
            x_t - (beta_t / self.sqrt_one_minus_alphas_cumprod[t]) *
        )

        # 노이즈 추가 (t=0일 때는 노이즈 없음)
        if t > 0:
            noise = torch.randn_like(x_t)
            sigma = torch.sqrt(beta_t)
            x_t_minus_1 = mean + sigma * noise
        else:
            x_t_minus_1 = mean
```

```
            x_t_minus_1 = mean

        return x_t_minus_1

    def _get_null_embedding(self, batch_size, device):
        """빈 조건(∅)에 대한 CLIP 임베딩 반환"""
        # 실제 구현에서는 빈 문자열 ""을 토크나이즈 후 CLIP에 통과
        # 여기서는 간략화를 위해 학습된 null embedding을 가정
        return torch.zeros(batch_size, 77, 768, device=device)


# === 배치 최적화된 CFG (실전 방식) ===
@torch.no_grad()
def cfg_forward_pass_batched(model, x_t, t, text_embedding, null_eml
                             guidance_scale=7.5):
    """
    실전에서는 2번의 forward pass를 배치로 묶어 1번에 처리한다.
    GPU 활용률을 높이기 위한 최적화.

    Args:
        model: 디퓨전 모델
        x_t: [B, C, H, W] 현재 노이즈 이미지
        t: [B] 타임스텝
        text_embedding: [B, 77, 768] 텍스트 조건
        null_embedding: [B, 77, 768] 빈 조건
        guidance_scale: w 값
    Returns:
        eps_guided: [B, C, H, W] CFG가 적용된 노이즈 예측
    """
    # 조건부와 무조건부 입력을 배치 차원으로 합치기
    x_t_combined = torch.cat([x_t, x_t], dim=0)          # [2B, C,
    t_combined = torch.cat([t, t], dim=0)                  # [2B]
    cond_combined = torch.cat([text_embedding, null_embedding], dim=

    # 단 1번의 forward pass로 두 예측을 동시에 수행
    eps_combined = model(x_t_combined, t_combined, cond_combined)  #

    # 조건부와 무조건부 예측 분리
    eps_cond, eps_uncond = eps_combined.chunk(2, dim=0)  # 각각 [B,

    # CFG 외삽
    eps_guided = eps_uncond + guidance_scale * (eps_cond - eps_uncor

    return eps_guided
```

⚠️ **계산 비용 참고**: CFG는 매 스텝마다 forward pass가 2배 필요하므로, 전체 생성 시간이 약 2배 증가한다. 배치 결합 방식을 사용하면 GPU 병렬성 덕분에 순차적 2회 호출보다 빠르다.

4.7 Negative Prompt

> 💡 **왜 Negative Prompt라는 개념이 필요한가?** 일반 CFG에서 무조건부 예측 $\epsilon_\theta(x_t, t, \varnothing)$은 "아무 이미지"를 향한 방향이다. 방향 벡터는 "아무 이미지 → 원하는 이미지"이다. 하지만 때로는 단순히 "원하는 이미지"를 지정하는 것만으로는 부족하다. "원하지 않는 것"을 명시적으로 지정해서 그 반대로 밀어내고 싶을 때가 있다. 예를 들어, "고화질 풍경"을 원하지만 "흐릿하고 워터마크가 있는" 결과가 자주 나온다면, 이런 원하지 않는 속성을 명시적으로 회피할 수 있으면 좋겠다.

Stable Diffusion의 Negative Prompt가 작동하는 원리:

$$\tilde{\epsilon} = \epsilon_\theta(x_t, t, c_{\text{neg}}) + w \cdot (\epsilon_\theta(x_t, t, c_{\text{pos}}) - \epsilon_\theta(x_t, t, c_{\text{neg}}))$$

빈 조건 $\varnothing$ 대신 **원하지 않는 조건** c_{neg}을 넣는다:

- c_{pos}: "a beautiful landscape, 4k, detailed"
- c_{neg}: "blurry, low quality, watermark"

방향 벡터가 "blurry에서 beautiful로"가 되므로, blurry를 적극적으로 회피하는 효과.

> 💡 **Negative Prompt의 수학적 원리: 왜 빼기가 "회피"가 되는가?**
>
> 일반 CFG에서 방향 벡터는:
>
> $$\vec{d}_{\text{standard}} = \epsilon_\theta(x_t, t, c_{\text{pos}}) - \epsilon_\theta(x_t, t, \varnothing)$$
>
> 이것은 "아무것도 아닌 것 → 원하는 것" 방향이다.
>
> Negative prompt CFG에서 방향 벡터는:
>
> $$\vec{d}_{\text{negative}} = \epsilon_\theta(x_t, t, c_{\text{pos}}) - \epsilon_\theta(x_t, t, c_{\text{neg}})$$
>
> 이것은 "원하지 않는 것 → 원하는 것" 방향이다.
>
> 이 방향으로 w배 외삽하므로, 결과는 c_{pos}에 가까울 뿐 아니라 c_{neg}에서 **적극적으로 멀어진다**. 벡터 공간에서 보면, 출발점(c_{neg})에서 목표점(c_{pos})을 잇는 직선 위에서 목표점을 넘어 w배 더 나아가는 것이다. 출발점이 "blurry"이고 도착점이 "beautiful"이면, 외삽은 "blurry의 정반대인 매우 선명한" 방향으로 이동한다.
>
> 이것은 스코어 함수 관점에서 보면, $\nabla_{x_t} \log p(c_{\text{pos}} \mid x_t) - \nabla_{x_t} \log p(c_{\text{neg}} \mid x_t)$ 방향으로 이동하는 것이다. 즉, c_{pos}의 확률은 높이면서 c_{neg}의 확률은 낮추는 방향이다.

4.8 PyTorch 구현: Negative Prompt

```python
@torch.no_grad()
def sample_with_negative_prompt(
    model,
    clip_text_encoder,
    positive_prompt: str,
    negative_prompt: str = "",
    guidance_scale: float = 7.5,
    num_timesteps: int = 1000,
    image_shape=(1, 4, 64, 64),
    device='cuda'
):
    """
    Negative Prompt를 지원하는 CFG 샘플링.

    일반 CFG에서 빈 조건(∅) 대신 negative prompt 임베딩을 사용한다.
    이를 통해 원하지 않는 속성을 적극적으로 회피할 수 있다.

    Args:
        model: 조건부 디퓨전 모델
        clip_text_encoder: CLIP 텍스트 인코더
        positive_prompt: 원하는 내용 (예: "a beautiful landscape, 4k'
        negative_prompt: 피하고 싶은 내용 (예: "blurry, low quality,
        guidance_scale: CFG 스케일 w
    Returns:
        x_0: 생성된 이미지
    """
    # 1. 텍스트 임베딩 생성
    # Positive prompt → 조건부 예측에 사용
    pos_tokens = tokenize(positive_prompt).to(device)
    c_pos = clip_text_encoder(pos_tokens)  # [1, 77, 768]

    # Negative prompt → 무조건부 예측 대신 사용
    # 빈 문자열이면 일반 CFG와 동일 (∅ 사용)
    neg_tokens = tokenize(negative_prompt).to(device)
    c_neg = clip_text_encoder(neg_tokens)  # [1, 77, 768]

    # 2. 노이즈에서 시작
    x_t = torch.randn(image_shape, device=device)

    # 3. 역방향 디노이징
    for t in reversed(range(num_timesteps)):
        t_tensor = torch.tensor([t], device=device)

        # 배치로 묶어서 한 번에 forward pass
        x_combined = torch.cat([x_t, x_t], dim=0)
        t_combined = torch.cat([t_tensor, t_tensor], dim=0)
        c_combined = torch.cat([c_pos, c_neg], dim=0)  # ← 여기가 핵

        eps_combined = model(x_combined, t_combined, c_combined)
        eps_pos, eps_neg = eps_combined.chunk(2, dim=0)

        # Negative Prompt CFG:
```

```
        # 방향 = "negative에서 positive로" → negative 속성을 적극 회피
        #
        # 일반 CFG:       ε̃= ε(∅)       + w * (ε(c_pos) - ε(∅))
        # Neg Prompt CFG: ε̃= ε(c_neg) + w * (ε(c_pos) - ε(c_neg))
        eps_guided = eps_neg + guidance_scale * (eps_pos - eps_neg)

        # DDPM 디노이징 스텝
        x_t = denoise_step(x_t, eps_guided, t)

    return x_t


# === 사용 예시 ===
# 기본 사용: 빈 negative prompt (일반 CFG와 동일)
image = sample_with_negative_prompt(
    model, clip_encoder,
    positive_prompt="a serene mountain landscape at sunset, oil pai
    negative_prompt="",  # = 일반 CFG
    guidance_scale=7.5
)

# Negative prompt 활용: 원하지 않는 요소 명시
image = sample_with_negative_prompt(
    model, clip_encoder,
    positive_prompt="a serene mountain landscape at sunset, oil pai
    negative_prompt="blurry, low quality, watermark, text, ugly, de
    guidance_scale=7.5
)

# 강한 스타일 유도: negative로 다른 스타일을 넣어 대비 효과
image = sample_with_negative_prompt(
    model, clip_encoder,
    positive_prompt="anime style portrait of a warrior",
    negative_prompt="realistic, photograph, 3d render",
    guidance_scale=9.0
)
```

⚠️ **실전 팁**: Negative prompt를 너무 길게 작성하면 효과가 분산될 수 있다. 핵심적으로 피하고 싶은 속성 3~5개를 간결하게 나열하는 것이 효과적이다.

4.9 Guidance Scale의 실용적 선택

용도	추천 w	이유
다양한 이미지 탐색	3~5	다양성 유지
프롬프트에 충실한 생성	**7~8** (가장 일반적)	품질-다양성 균형
정밀한 텍스트 따르기	10~15	포화 주의
스타일 전이, 일러스트	5~12 (스타일에 따라)	실험 필요

> 💡 **왜 7~8이 "마법의 숫자"인가?** 이론적으로 최적의 w를 계산하는 방법은 없다 -- 순전히 경험적 발견이다. 하지만 직관적 설명은 가능하다: $w = 1$에서 모델의 조건부 예측은 학습 데이터의 노이즈와 불확실성 때문에 "충분히 조건을 따르지 않는다." $w = 7 \sim 8$은 이 과소적합을 보상하기에 적절한 증폭 수준이다. w가 이 범위를 넘어가면 보상이 아니라 과보상이 되어 아티팩트가 발생한다.
>
> 실제로 최적의 w는 모델, 데이터셋, 해상도, 샘플러에 따라 다르다. SDXL은 $w = 5 \sim 7$이 좋고, SD 1.5는 $w = 7 \sim 9$가 좋은 경향이 있다. 새로운 모델을 사용할 때는 w를 변화시키며 실험하는 것이 중요하다.

---

5. 조건부 학습의 전체 파이프라인

> 💡 **이 절에서는 지금까지 배운 모든 구성 요소가 어떻게 하나의 시스템으로 결합되는지 보여준다.** CLIP (Section 1) → 텍스트를 임베딩으로 변환, Cross-Attention (Section 2) → 임베딩을 모델에 주입, 조건 드롭아웃 (Section 4.2) → CFG를 위한 학습 준비, DDPM (강의 7) → 실제 디노이징 학습. 이 모든 것이 하나의 학습 루프에 통합된다.

5.1 학습 시

```
1. 텍스트-이미지 쌍 (text, x₀) 로드
2. text → [CLIP Text Encoder] → 조건 임베딩 c
3. 10% 확률로 c → ∅ (빈 조건) 교체
4. t ~ Uniform{1,...,T}, ε ~ N(0,I)
5. x_t = √ᾱ_t · x₀ + √(1-ᾱ_t) · ε
6. ε^= ε_θ(x_t, t, c)       # 조건 c를 Cross-Attention으로 주입
7. Loss = ‖ε - ε‖²
```

> 💡 **각 단계가 왜 필요한가:**
>
> - **Step 2**: CLIP 텍스트 인코더는 보통 **고정(frozen)**된다. 이미 4억 개의 데이터로 학습된 표현을 그대로 사용하는 것이 더 효율적이다. 디퓨전 모델만 학습한다.
> - **Step 3**: CFG의 핵심. 이 단순한 조건 드롭아웃이 없으면, 추론 시 무조건부 예측 $\epsilon_\theta(x_t, t, \varnothing)$의 품질이 떨어져서 CFG가 제대로 작동하지 않는다.
> - **Step 5**: Forward process. 원본 이미지 x_0에 t 단계만큼의 노이즈를 추가한다. 이때 실제로 $t-1, t-2, \ldots$ 단계를 순차적으로 거치지 않고, 닫힌 형태(closed-form)로 한 번에 계산한다.
> - **Step 7**: 모델이 예측한 노이즈 $\hat{\epsilon}$과 실제 추가된 노이즈 ϵ의 MSE. 이 손실을 최소화하면 모델은 "이미지에 섞인 노이즈를 정확하게 예측"하는 능력을 학습한다.

아래는 위 파이프라인의 PyTorch 구현이다:

```python
def train_conditional_diffusion(model, clip_encoder, dataloader,
                                num_epochs=100, p_uncond=0.1,
                                num_timesteps=1000, device='cuda')
    """
    CFG를 위한 조건부 디퓨전 모델 학습.

    핵심: p_uncond 확률로 조건을 빈 조건(∅)으로 교체하여,
    같은 모델이 조건부/무조건부 예측을 모두 학습하게 한다.
    """
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4)

    # 노이즈 스케줄 사전 계산
    betas = torch.linspace(1e-4, 0.02, num_timesteps).to(device)
    alphas = 1.0 - betas
    alphas_cumprod = torch.cumprod(alphas, dim=0)

    # 빈 조건 임베딩 사전 계산 (빈 문자열 "")
    with torch.no_grad():
        null_tokens = tokenize("").to(device)
        null_embedding = clip_encoder(null_tokens)  # [1, 77, 768]

    for epoch in range(num_epochs):
        for images, texts in dataloader:
            images = images.to(device)  # [B, 3, 256, 256] 또는 잠재

            # Step 2: 텍스트 → CLIP 임베딩
            with torch.no_grad():
                tokens = tokenize(texts).to(device)
                c = clip_encoder(tokens)  # [B, 77, 768]

            # Step 3: 조건 드롭아웃 (CFG 학습의 핵심!)
            # 각 샘플에 대해 독립적으로 p_uncond 확률로 빈 조건 사용
            mask = torch.rand(len(images), device=device) < p_uncon
            # mask가 True인 샘플은 빈 조건으로 교체
            c[mask] = null_embedding.expand(mask.sum(), -1, -1)

            # Step 4: 랜덤 타임스텝과 노이즈 샘플링
            t = torch.randint(0, num_timesteps, (len(images),), dev
            noise = torch.randn_like(images)

            # Step 5: Forward process - 노이즈 추가
            sqrt_alpha_bar = torch.sqrt(alphas_cumprod[t])[:, None,
            sqrt_one_minus_alpha_bar = torch.sqrt(1 - alphas_cumpro
            x_t = sqrt_alpha_bar * images + sqrt_one_minus_alpha_ba

            # Step 6: 모델 예측
            predicted_noise = model(x_t, t, c)

            # Step 7: MSE Loss
            loss = F.mse_loss(predicted_noise, noise)

            optimizer.zero_grad()
```

```
optimizer.zero_grad()
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
optimizer.step()
```

5.2 생성 시

```
1. 텍스트 → [CLIP Text Encoder] → c
2. x_T ~ N(0, I)
3. for t = T, ..., 1:
     ε_uncond = ε_θ(x_t, t, ∅)       # 무조건부 예측
     ε_cond   = ε_θ(x_t, t, c)       # 조건부 예측
     ε̃= ε_uncond + w·(ε_cond - ε_uncond)  # CFG
     x_{t-1} = denoise_step(x_t, ε̃, t)
4. return x₀
```

> **주의**: 생성 시 매 스텝마다 모델을 **2번** 호출해야 한다 (조건부 + 무조건부). CFG의 계산 비용은 2배.

> 💡 **학습 vs 생성의 핵심 차이 요약**
>
> | 측면 | 학습 시 | 생성 시 |
> |---|---|---|
> | 모델 호출 횟수/스텝 | 1번 (조건부 또는 무조건부 중 하나) | 2번 (조건부 + 무조건부) |
> | 방향 | 랜덤 t에서 노이즈 예측 학습 | T부터 0까지 순차적 디노이징 |
> | CLIP 인코더 | 조건 임베딩 c 생성 (고정) | 조건 임베딩 c + 빈 조건 $\varnothing$ 모두 필요 |
> | CFG 역할 | 조건 드롭아웃으로 준비만 함 | 실제 외삽으로 품질 향상 |
>
> 학습 시에는 CFG가 직접 사용되지 않는다 -- 단지 모델이 $\varnothing$도 처리할 수 있도록 준비해놓을 뿐이다. CFG의 마법은 생성 시에만 발휘된다.

---

6. FID와 CLIP Score의 Trade-off

> 💡 **왜 두 가지 지표를 모두 봐야 하는가?** 이미지 생성 모델의 품질을 단일 숫자로 평가하기는 어렵다. 두 가지 다른 측면을 측정해야 하기 때문이다:
>
> - **FID (Frechet Inception Distance)**: 생성된 이미지의 **현실성과 다양성**을 측정한다. 생성 이미지의 분포와 실제 이미지의 분포 사이의 거리를 계산한다. FID가 낮을수록 좋다. 하지만 FID는 "텍스트 조건에 얼마나 충실한가"는 측정하지 않는다 -- 아무 고품질 이미지나 만들어도 FID가 좋을 수 있다.

> - **CLIP Score**: 생성된 이미지와 입력 텍스트의 **의미적 일치도**를 측정한다. CLIP 모델로 이미지 임베딩과 텍스트 임베딩의 코사인 유사도를 계산한다. CLIP Score가 높을수록 텍스트에 충실하다. 하지만 CLIP Score만 높아도 이미지가 자연스럽지 않을 수 있다.

6.1 Guidance Scale에 따른 지표 변화

```
FID (↓)     CLIP Score (↑)
  |            |
  |   \        |        /
  |    \       |       /
  |     \      |      /
  |      \ best |     /
  |       \————|————/
  |        \   |   /
  |         \  |  /
  |          \ | /
  └————————————|————————→ w (guidance scale)
  0    3    7   10   15
```

- w 증가 → CLIP Score 증가 (텍스트에 더 충실)
- w 증가 → FID는 처음에 개선되다가 과도하면 악화 (다양성 감소, 포화)
- **최적점**: 보통 $w = 7 \sim 8$ 부근

> 💡 **FID가 U자 곡선을 보이는 이유** w가 작을 때($w < 3$): 생성 이미지가 너무 다양하고 초점이 없어서, 실제 데이터 분포와의 거리(FID)가 크다. 조건을 무시한 "아무 이미지"가 많이 섞여 있기 때문이다.
>
> w가 중간일 때($w \approx 7$): 조건에 충실하면서도 적절한 다양성을 유지한다. 생성 이미지의 분포가 실제 데이터의 조건부 분포 $p(x \mid c)$에 가장 가까운 영역이다. FID가 최소가 된다.
>
> w가 클 때($w > 12$): 모든 이미지가 "가장 전형적인" 하나의 모드로 수렴한다. 다양성이 극단적으로 줄어들고, 색상이 포화되고 아티팩트가 생겨 FID가 다시 악화된다. 생성 분포가 실제 데이터 분포보다 훨씬 좁아진 것이다.

6.2 Pareto Front

$$\text{FID} \leftrightarrow \text{CLIP Score (또는 IS)}$$

어떤 w를 선택하든 한쪽이 개선되면 다른 쪽이 악화되는 **trade-off**가 존재. CFG는 이 Pareto front를 직접 탐색할 수 있게 해준다.

> 💡 **Pareto front란 무엇인가?** 다목적 최적화(multi-objective optimization)에서 Pareto front는 "어떤 목표도 다른 목표를 희생하지 않고는 더 개선할 수 없는 해들의 집합"이다. FID-CLIP Score 평면에서 w를 변화시키면 하나의 곡선이 그려지는데, 이 곡선의 왼쪽 아래 경계(FID 낮고 CLIP Score 높은 쪽)가 Pareto front이다. CFG의 실용적 가치는, w 하나의 파라미터만 조절하면 이 Pareto front 위의 원하는

> 지점을 선택할 수 있다는 점이다. w가 없으면 모델을 재학습해야 하지만, w는 추론 시에 자유롭게 변경할 수 있다.

---

7. 조건부 생성의 다른 접근법들

> 💡 **왜 다양한 조건 주입 방법이 존재하는가?** "조건"이라는 것 자체가 다양한 형태를 가지기 때문이다. 텍스트("고양이를 그려줘")는 의미적 조건이고, 엣지 맵이나 포즈는 공간적 조건이며, 참조 이미지는 스타일 조건이다. 각 형태의 조건에 최적화된 주입 방법이 다르다. CFG는 이 중에서 가장 범용적인 방법으로, 조건의 종류에 관계없이 적용할 수 있다.

7.1 비교

방법	별도 모델 필요?	범용성	품질
클래스 조건 (class label)	없음	제한적 (ImageNet 클래스)	좋음
Classifier Guidance	분류기 필요	클래스 기반	좋음
Classifier-Free Guidance	없음	**범용 (텍스트, 이미지 등)**	**최고**
ControlNet	추가 인코더	공간적 조건 (포즈, 엣지)	좋음
IP-Adapter	추가 어댑터	이미지 스타일 조건	좋음

> 💡 **이 표에서 읽어야 할 핵심 트렌드**: 조건부 생성 기술의 발전은 "범용성 증가"와 "추가 모델 불필요" 방향으로 진행되어 왔다. 클래스 조건 → Classifier Guidance → CFG로 가면서, 점점 더 적은 추가 구성 요소로 더 넓은 범위의 조건을 처리할 수 있게 되었다. ControlNet과 IP-Adapter는 CFG와 경쟁하는 것이 아니라, CFG와 **결합**되어 서로 다른 종류의 조건을 동시에 처리한다.

7.2 ControlNet (간략)

> 📄 Zhang et al., "Adding Conditional Control to Text-to-Image Diffusion Models", ICCV 2023

추가적인 공간 조건(엣지, 포즈, 깊이 맵)을 주입:

```
텍스트 조건 → [Cross-Attention으로 U-Net에 주입]
공간 조건 → [ControlNet Encoder] → [Zero-Conv로 U-Net에 더함]
```

CFG와 직교적: CFG는 텍스트 조건의 **강도**를 조절하고, ControlNet은 **공간적 구조**를 조건으로 추가.

> 💡 **"직교적(orthogonal)"의 의미**: CFG와 ControlNet은 서로 독립적으로 작동한다는 뜻이다. CFG는 "무엇을 그릴까"(의미적 내용)의 충실도를 조절하고,

> ControlNet은 "어디에 그릴까"(공간적 배치)를 제어한다. 따라서 ControlNet을 사용하면서 동시에 CFG의 guidance scale w를 조절할 수 있다. 두 기법은 서로 간섭하지 않고 보완적으로 작동한다. 실전에서는 "Canny 엣지 맵으로 구도를 잡고, 텍스트 프롬프트로 내용을 지정하며, w로 충실도를 조절하는" 3단계 제어가 가능하다.

---

8. 핵심 수식 요약

이름	수식
CLIP Loss	$\mathcal{L} = -\frac{1}{N}\sum_i \log \frac{\exp(S_{ii})}{\sum_j \exp(S_{ij})}$
Classifier Guidance	$\hat{\epsilon} = \epsilon_\theta(x_t) - s\sqrt{1-\bar{\alpha}_t}\nabla_{x_t} \log p_\phi(y\|x_t)$
CFG	$\tilde{\epsilon} = \epsilon_\theta(x_t, \varnothing) + w(\epsilon_\theta(x_t, c) - \epsilon_\theta(x_t, \varnothing))$
Cross-Attention	$\mathrm{Attn}(Q_{\mathrm{img}}, K_{\mathrm{txt}}, V_{\mathrm{txt}})$
Negative Prompt CFG	$\tilde{\epsilon} = \epsilon_\theta(x_t, c_{\mathrm{neg}}) + w(\epsilon_\theta(x_t, c_{\mathrm{pos}}) - \epsilon_\theta(x_t, c_{\mathrm{neg}}))$

---

9. 다음 강의 미리보기

지금까지의 조각들을 결합하면 **Stable Diffusion**이 된다:

- **VQ-GAN** (강의 4) → 이미지를 잠재 공간으로 압축
- **DDPM** (강의 7) → 잠재 공간에서 디퓨전
- **CLIP** (이번 강의) → 텍스트 조건 인코딩
- **CFG** (이번 강의) → 조건 강화

> 💡 **이번 강의의 핵심 관계를 되짚어보기** 이번 강의에서 다룬 기법들의 관계를 정리하면:
>
> 1. **CLIP**은 텍스트와 이미지 사이의 **의미적 다리(semantic bridge)**를 만든다. Contrastive learning으로 학습된 공유 임베딩 공간이 이 다리의 핵심이다.
> 2. **Cross-Attention**은 CLIP이 만든 텍스트 임베딩을 디퓨전 모델에 **공간 선택적으로** 주입하는 메커니즘이다.
> 3. **CFG**는 이 주입된 조건의 **영향력을 증폭**하여, 조건에 더 충실한 이미지를 생성하게 한다.
> 4. **Negative Prompt**는 CFG의 확장으로, 원하지 않는 속성을 **적극적으로 회피**하게 한다.
>
> 이 네 가지가 없으면 현대의 text-to-image 생성 시스템은 존재할 수 없다.

다음 강의에서는 이들을 합쳐 **Latent Diffusion Model / Stable Diffusion**을 다루고, 이후 **DiT**, **Flow Matching**으로 진행한다.

참고문헌 (References)

1. Radford, A., Kim, J. W., Hallacy, C., et al. (2021). **"Learning Transferable Visual Models From Natural Language Supervision."** *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*. OpenAI. arXiv:2103.00020
2. Ho, J. & Salimans, T. (2022). **"Classifier-Free Diffusion Guidance."** *NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications*. arXiv:2207.12598
3. Dhariwal, P. & Nichol, A. (2021). **"Diffusion Models Beat GANs on Image Synthesis."** *Advances in Neural Information Processing Systems (NeurIPS 2021)*. arXiv:2105.05233
4. Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). **"High-Resolution Image Synthesis with Latent Diffusion Models."** *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2022)*. arXiv:2112.10752
5. Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). **"Hierarchical Text-Conditional Image Generation with CLIP Latents."** (DALL-E 2). arXiv:2204.06125
6. Peebles, W. & Xie, S. (2023). **"Scalable Diffusion Models with Transformers."** *IEEE/CVF International Conference on Computer Vision (ICCV 2023)*. arXiv:2212.09748
7. Zhang, L., Rao, A., & Agrawala, M. (2023). **"Adding Conditional Control to Text-to-Image Diffusion Models."** *IEEE/CVF International Conference on Computer Vision (ICCV 2023)*. arXiv:2302.05543
8. Esser, P., Kulal, S., Blattmann, A., et al. (2024). **"Scaling Rectified Flow Transformers for High-Resolution Image Synthesis."** (Stable Diffusion 3). arXiv:2403.03206